

Winterborn Restock System — Development Brief

Internal document for the build team. Not client-facing. Hours only, no pricing.

Contract accepted: Aug 18 · Feature-complete: Sept 11 · Pilot live: Sept 18 · Stage 2: late Oct · Budget: 138 hrs total

1. What we are building

A single-source-of-truth inventory and restock system for a seasonal retailer (14 US Christmas-market locations, ~\$2.9M season, 41k transactions Nov-Dec). Square POS stays the till and sales record. Our system owns everything else: warehouse intake, product catalog discipline, restock requests, box-level dispatch, per-location stock counts, write-offs, and season-close reconciliation. Sortly (their current warehouse app) is retired to read-only; its export (564 items, 42,428 units, full colour/size attributes) is already parsed and becomes our seed data.

Core ledger principle: inventory is never a stored number, it is a derivation:

$$\text{on_hand}(\text{variation}, \text{location}) = \Sigma \text{dispatched} - \Sigma \text{sold} - \Sigma \text{written_off}$$

Dispatches live in our DB (we create them). Sales live in Square permanently (Orders API). Therefore any count can be recomputed from scratch at any time — a missed webhook or a bad write can never cause permanent drift. Build everything around replayability.

Hard constraint from the client's physical reality: market sites are chaotic (staging piles, staff grabbing stock freely, unsold items tossed in back). **Nothing is ever scanned or counted at a market.** All discipline lives at ONE warehouse. Market staff behaviour changes in exactly one way: tapping a colour at the till (behaviour they already have on one product line).

2. Architecture

- **Frontend:** PWA (React), installable to home screen, phone-first. Roles: Owner, Warehouse, Market Manager, Operator (us).
- **Backend:** Node/API server + Postgres. All state transitions are append-only events where practical (dispatch, receive-sale, write-off, intake) so the ledger derivation above is trivial.
- **Square integration:** read-mostly.
 - Catalog API: create/update items, variations, SKUs (Stage: catalog restructure).
 - Webhooks: `order.created` / `payment.updated` → ingest sale lines (variation id, qty, location,

timestamp) into our DB within seconds. Verify Square signature on every event.

- **Reconciliation poll:** every 15–30 min, Orders search per location since last cursor. The poll is the source of truth; webhooks are only the low-latency trigger. Design so a week of missed webhooks self-heals on one poll pass.
- We do NOT write inventory into Square in v1. (Optional Stage 2 flag: one-way mirror pushing derived counts into Square’s inventory module for owner familiarity. Decision deferred; build nothing for it in Stage 1.)
- **Hosting:** any managed Node+Postgres host; HTTPS required (webhooks). Region: US-east preferred (client + Square latency).
- **Auth:** email magic-link or simple credentialed login; no SSO needed. Keep it boring.

3. The colour model (two tiers) — get this right, everything depends on it

Tier 1 — Till families. Curated per category (e.g. Blue, Gray, Pink, Cream, Multi). These become the Square variations the cashier taps. Rule for choosing: any staffer must be able to pick correctly by *looking at the item in hand* mid-queue. Target 6–12 families per category, decided with the warehouse lead (Casey) — pending session, do not hard-code assumptions.

Tier 2 — Warehouse variants. The full detail (245 distinct colour names in the Sortly seed: “Wine & Smoke”, “French Gray”, ...). Each variant maps to exactly ONE family. Manifests, intake, pick lists, and season-close reconciliation run at variant level. Sales run at family level.

Vocabulary enforcement (intake UI): colour is never free-typed. Autocomplete over existing variants; creating a new variant runs a fuzzy match (“3 similar exist: Navy, Night Blue, Denim — use one?”) and forces family assignment at creation. This is the mechanism that stops “blue / light blue / navy blue” drift — treat it as a first-class feature, not polish.

Precision map (what we promise, where): variant-exact at intake, in warehouse stock, on manifests, at dispatch, and at season-close scan-back; family-accurate at live market level (sales decrement by family). Sent-minus-returned at season close recovers variant-level sell-through per market. Do not attempt variant precision at the till; forced precision there produces confidently-wrong data.

4. Catalog restructuring + SKUs (the risky work — do the prototype FIRST)

Current Square state: ~74 catalog rows; Scarves = 14 flat items (no variations); Mittens = style variations only; Headwear already has colour variations (the model to copy); zero SKUs anywhere; shared catalog account-wide with per-location availability flags and per-location price overrides.

The trap: converting a FLAT item to a variation-structured item is a restructure, not an edit. Done wrong, last season’s sales history orphans and year-over-year reporting breaks — which would destroy the headline value of the project.

Mandatory protocol: 1. **Hour 1-3 of the project: prototype on ONE low-volume flat item** in sandbox, then one live item. Verify: historical order lines still resolve to the item; Item Sales reports still aggregate correctly; the new variations sell correctly on a test device. 2. Preferred approach: add variations TO the existing item object (preserving `item_id`), with the existing single price point becoming per-variation prices; avoid create-new-and-archive-old unless the prototype proves it's required. 3. Only after sign-off on the prototype: bulk-run Scarves (29% of revenue, first), then Mittens, Socks, Stuffies, Capes/Wraps. Script the mapping from the parsed Sortly attributes. 4. **Per-location price overrides must survive.** Cape runs \$165–\$177 by market; any catalog write must preserve location overrides. Test explicitly. 5. **Catalog freeze Sept 12.** Variations go live account-wide on the shared catalog; the trial market must open on a finished catalog, not an iterating one.

SKU scheme: CAT-GROUP-COLOR-SIZE style (e.g. SCF-STR-WNE-R), generated once from the Sortly parse, written to every Square variation and stored in our DB as the join key. Include a brand segment for the [Fraas] resale line (Carmel only, ~\$10k). Supplies (bags etc.) get SKUs but are flagged non-sale. Collision-check across all 564 items.

5. Feature modules

5.1 Warehouse intake (Sortly replacement)

Form: item (search-as-you-type) → variant (controlled vocab, family auto-attached) → qty → unit cost → photo (phone camera). Target: under 60 seconds per line. Every intake is a ledger event (+warehouse stock). Cost capture matters: it unlocks margin reporting later (cost is blank on 558/564 seed items).

5.2 Restock requests

Raised three ways: auto-draft from threshold engine, from the Monday review, or manually. A request = market + lines (family or variant level + qty). Warehouse queue view; both sides can edit before packing; every edit logged (who/when/old→new). States: draft → open → packing → dispatched → (optional: arrived) → closed.

5.3 Pack, label, dispatch

Packer resolves family quantities to concrete variants ("60 gray" → 40 Charcoal + 20 Ash), recorded on the manifest. One QR label per box, printed from the tool via browser print to a thermal printer (Brother QL-class, ~\$100 — client purchasing; needs to be at warehouse by Sept 15). **QR encodes box ID only** — contents live in the DB, so a manifest edited pre-dispatch never orphans its label. Dispatch scan at the warehouse door posts the manifest to the destination location's ledger (+sent). Counts post at dispatch by default; an optional one-tap "arrived" state exists for in-transit visibility but is not required for the math and nothing at the market depends on it.

5.4 Load verification

Loader selects vehicle/destination, scans boxes on. Box destined elsewhere → immediate error. Cheap feature (scan infra already exists), high value — catches wrong-van errors nobody currently detects until a market opens the box.

5.5 Live sales sync

Webhook ingest + reconciliation poll as per Section 2. Each sale line decrements sold for (family-variation, location). Latency target: seconds via webhook, hard-guaranteed within one poll cycle. Log every ingested event with source (webhook|poll) for debugging.

5.6 Threshold engine

Per (variation, location) min levels. Seed defaults from last season's style-level velocity (2025 exports are parsed); owner/operator can edit. On each ledger change: below threshold → auto-draft restock request (deduped: don't stack drafts for the same line). **Stage 1: thresholds exist but alerts are manual-review only (operator watches dashboard daily). Stage 2: automated alerts on.**

5.7 Write-off flow

Reasons: damage / gift / sample. Ledger event (–stock at location or warehouse). Replaces their “100% Gifted/Destroyed” POS discount (74 uses last season) — that discount gets retired at Stage 2, not by us deleting it but by the owner once the flow is adopted.

5.8 Dashboard (Owner + Operator views)

Per market: on-hand by family (derived), sales today/this week by family, open/in-transit requests, variance log. Account-wide: low-stock list across all markets, decision queue. Keep it one screen; the audit's Monday-review / Friday-dispatch cadence is the workflow it serves.

5.9 Season-close reconciliation

Leftover boxes scan back at the warehouse (variant-level manifests on return). Output: sent – returned = variant-level sell-through per market — the first colour-level demand data this business has ever had, and the seed for 2027 prediction features. Stage 2 feature; design the data model for it from day one (returns are just negative dispatches).

6. Hour budget (138 total)

MODULE	HRS	STAGE
Flat-item migration prototype + sign-off	3	1
Catalog restructure (5 categories, scripted from Sortly parse) + till family sets	18	1
SKU generation + write to Square + collision checks	9	1
Core app: auth, roles, request workflow, edit logging	26	1
Pack/label/dispatch + QR + load verification	15	1
Square sync: webhooks, signature verify, reconciliation poll, ledger	17	1
Threshold engine (manual-review mode) + dashboard v1	10	1
Pilot deploy, opening-load support, on-market watch (Sept 15–20)	8	1
Stage 1 subtotal (feature-complete Sept 11)	106	
Warehouse intake module (vocab enforcement, photos, costs)	12	2
Write-off flow	4	2
Automated alerts, notifications, dashboard hardening from pilot findings	8	2
All-markets rollout, training, Sortly retirement, docs	8	2
Stage 2 subtotal (late Oct)	32	
Total	138	

Buffer reality: 106 hrs in 24 days is ~4.5 hrs/day. Anything trending over budget gets flagged at the 70% mark of its line, not after.

Note: warehouse intake is Stage 2 in the hour budget, but the Sept 15–17 opening load still has to be entered — for the pilot, opening stock is loaded via the dispatch/manifest flow (which exists in Stage 1) or a one-off scripted import from the Sortly parse. Do not pull the full intake UI forward for this.

7. Timeline & gates

DATE	GATE
Aug 18-19	Prototype flat-item migration (sandbox → 1 live item). Blocker gate: nothing else in catalog work proceeds without sign-off.
Aug 18-20	Market locations + tax-rate setup (client-requested, 14 locations, per-state rates). Chase Casey session. Client orders printer. Obtain production API token from owner (owner-only console — walk him through app creation + token with Catalog/Inventory/Orders/Merchants scopes).
Aug 25	Catalog restructure + SKUs complete in sandbox; begin production run
Sept 1	Core app + sync working end-to-end in staging against sandbox
Sept 8	Production dry run: real catalog, test dispatches
Sept 11	Feature-complete. One-week buffer begins — no new features after this date, period.
Sept 12-17	Opening load packed/labelled through the system; printer live; staff trained (10 min: colour tap only)
Sept 18	Pilot live at trial market. Operator watches daily through opening weeks.
Week of Sept 22	Pilot review: till-accuracy spot checks, sync lag, count drift, decision load
Late Oct	Stage 2 complete; all 14 markets ready; Sortly read-only
Nov	Season live
Jan	Season-close scan-back + sell-through report

8. Known risks & standing decisions

- **Sept 12 catalog freeze is absolute** — shared catalog means changes hit all locations; the pilot market must open on a frozen catalog.
- **Opening-load rule:** everything entering the pilot market goes through a labelled box; the opening scans ARE the opening inventory. If anything is side-loaded, counts are fiction from hour one — this is a client-behaviour rule, enforce it in training and say it in writing.
- **Casey (warehouse lead) is a dependency** for family sets and intake design. If unscheduled by Aug 21, design from the Sortly export and validate later — do not let her calendar eat the buffer.
- **API token is owner-gated** (Square Developer Console is owner-only). Until received, everything runs against our own sandbox. Escalate if not in hand by Aug 22.
- **Counts are directional, not forensic** — by design. Mis-taps and shrinkage cause drift; spot checks + season-close reconciliation are the correction mechanisms. Never promise unit-perfect live counts to anyone.
- **No colour-level prediction this season** — 2025 data has colour on only 4.8% of revenue. Style-level pick-list suggestions are fine; colour prediction is a 2027 feature fed by this season's data. Do not build or imply it now.

- **Scope guard:** anything not in this document is an add-on conversation, not a quiet extension. The hour budget has no fat in it.